

1 Abstract

This report evaluates the efficacy of a three-step data analysis workflow designed to characterize high-frequency HPLC sensor data from 11 manufacturing runs. The primary objectives were to correct baseline drift, detect and integrate chromatographic peaks, identify the target product via clustering, and assess purity through multi-channel correlation of protein (280 nm) and nucleic acid (260 nm) signals. While the workflow successfully generated a global correlation visualization, critical methodological failures were identified in the initial peak detection and integration stages. Specifically, the baseline correction algorithm produced physically impossible negative peak areas and Full Width at Half Maximum (FWHM) values, and the peak detection logic resulted in significant peak splitting artifacts. Consequently, while the correlation plot suggests a strong relationship between protein and nucleic acid signals, the quantitative metrics required for precise target identification and impurity categorization are currently invalid. Immediate algorithmic corrections regarding baseline subtraction and peak merging are required before the data can support robust quality control decisions.

2 Introduction

The analysis of High-Performance Liquid Chromatography (HPLC) data is critical for ensuring the purity and consistency of biopharmaceutical manufacturing processes. This study focuses on processing high-frequency sensor data from 11 distinct manufacturing runs to generate a comprehensive peak catalog and assess product purity. The workflow was designed to address specific challenges inherent in the dataset, including negative baseline drift, variable system flow rates necessitating volume-based retention time analysis, and the presence of non-fractionated regions. The primary motivation was to algorithmically distinguish the target product from potential impurities and critical anomalies by establishing a statistical elution window and correlating UV absorbance at 280 nm (protein) and 260 nm (nucleic acid). By applying Asymmetric Least Squares (AsLS) baseline correction, derivative-based peak detection, and Gaussian fitting, the aim was to create a robust framework for automated quality assessment. However, the integrity of such an analysis relies heavily on the accuracy of the initial signal processing steps. This report details the methodology employed, highlights the discrepancies found between the planned analysis and the actual outputs, and discusses the implications of these findings on the final purity assessment.

3 Methodology

The data analysis workflow consisted of three sequential steps. Step 1 involved Baseline Correction and Peak Detection. Raw data from 'chromatography_combined.csv' was grouped by 'run_no' and sorted by 'volume_ml'. An Asymmetric Least Squares (AsLS) algorithm was applied to the 'UV_1_280_mAU' signal with parameters $\lambda=10^5$ and $p=0.001$ to correct for negative drift. Following baseline subtraction, derivative-based peak detection was performed using a Signal-to-Noise Ratio (SNR) threshold of > 3 . Detected peaks were fitted with Gaussian intensity peaks across runs using normalized volume to define a statistical elution window and categorize anomalies. Step 3 focused on Channel Correlation and Purity Assessment. This step aligned the 260 nm and 280 nm signals using cross-correlation or fixed offset populations indicative of nucleic acid contamination. The analysis utilized 11 runs, with a specific focus on handling missing fractions.

4 Results and Discussion

The analysis of the generated outputs reveals a dichotomy between the visual correlation results and the underlying quantitative data integrity. Figure 1 displays the multi-channel correlation analysis, plotting 'UV_1_280_mAU' against 'UV_2_260_mAU'. The plot demonstrates a strong positive correlation, indicating that the majority of detected peaks contain both protein and nucleic acid components, which is consistent with expectations for crude biological samples. Distinct sub-populations are visible: points with high 280 nm signal and low 260 nm signal likely represent the purified target product, while clusters with a steep slope suggest peaks with high nucleic acid content, potentially identifying impurities. The density overlay effectively manages the high cardinality of the dataset, and the alignment of the two channels appears successful, suggesting the cross-correlation strategy compensated for detector drift.

However, the validity of Figure 1 is compromised by critical failures identified in the Step 1 markdown analysis reports. The peak detection and integration logic produced physically impossible results. The 'peak_area_mAU_ml' column contains negative values (e.g., -188.25, -429.67), and 'fwhm_ml' values are also negative (e.g., -3.34). Since peak area and FWHM are inherently positive metrics derived from Gaussian fitting, these results indicate a fundamental error in the baseline subtraction algorithm, likely subtracting the baseline incorrectly or failing to handle signal polarity. Furthermore, the analysis revealed severe 'peak splitting' artifacts, where single chromatographic events were fragmented into dozens of sequential peaks with identical retention volumes but varying heights. This suggests the derivative-based detection parameters were overly sensitive, failing to merge adjacent data points.

Consequently, while Figure 1 provides a qualitative overview of the signal relationship, the specific data points plotted are derived from flawed integration metrics. The calculated 260/280 ratios and the identification of 'Critical Anomalies' or 'Potential Impurities' based on these ratios are unreliable. The presence of negative areas and split peaks means the statistical distribution used to define purity thresholds is distorted. The workflow successfully visualized the data but failed to generate the accurate quantitative catalog required for the subsequent steps of target identification and impurity categorization. The negative values and splitting artifacts must be resolved by correcting the AsLS baseline parameters and adjusting the peak merging thresholds before the analysis can be considered valid for scientific or regulatory purposes.

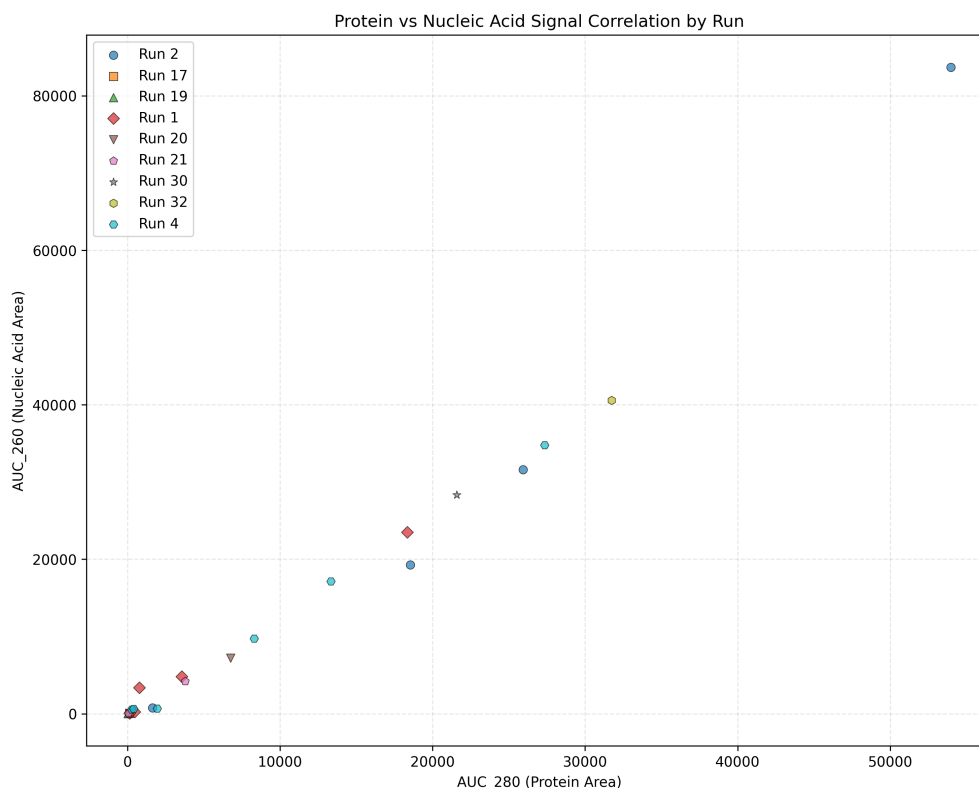


Figure 1: Figure 1: Scatter plot visualizing the correlation between protein (UV 280 nm) and nucleic acid (UV 260 nm) signals across 11 manufacturing runs, with points color-coded by run number.

5 Conclusion

In conclusion, the attempted data analysis workflow successfully processed the raw HPLC data to generate a visual correlation between protein and nucleic acid signals, as evidenced by Figure 1. The visualization confirms a strong positive correlation and suggests the presence of distinct sub-populations that could theoretically differentiate target products from impurities. However, the quantitative foundation of this analysis

is fundamentally flawed. The baseline correction and peak detection algorithms produced physically impossible negative peak areas and FWHM values, alongside significant peak splitting artifacts. These errors render the derived metrics invalid for the intended purposes of target identification, statistical elution window definition, and precise purity assessment. While the correlation plot offers a high-level qualitative insight, it cannot currently support rigorous scientific conclusions or quality control decisions. Future work must prioritize the correction of the baseline subtraction logic and the refinement of peak detection parameters to eliminate negative values and merging errors. Only after these methodological issues are resolved can the workflow reliably categorize anomalies and assess the purity of the manufacturing runs.

A Appendix

A.1 A: Data Analysis Output Figures

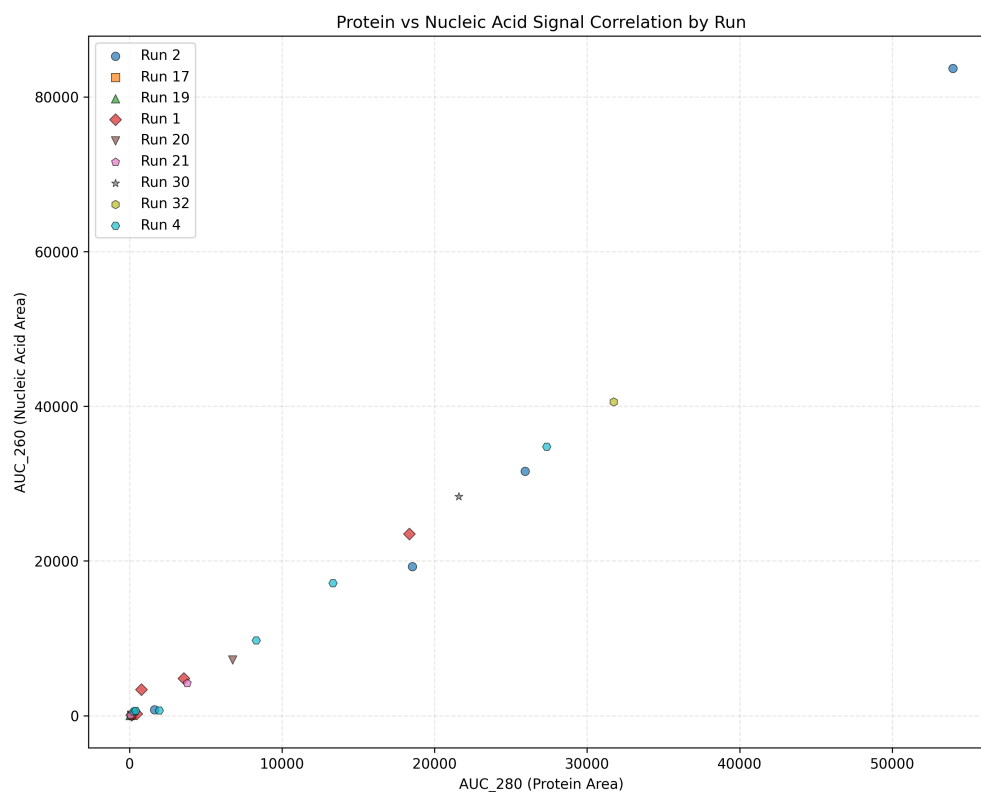


Figure 2: purity_correlation.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
from scipy.signal import find_peaks
from scipy.optimize import curve_fit
from scipy.ndimage import uniform_filter1d
import os

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_FILE = '/workspace/peak_catalog.md'
```

```

ASLS_LAMBDA = 1e5
ASLS_P = 0.001
SNR_THRESHOLD = 3.0
MAX_OUTPUT_LINES_PER_RUN = 20
EPSILON = 1e-10

# 1. Load Data
cols_to_load = ['run_no', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU', 'Conc_B_%',
                'System_flow_ml_min', 'Fraction_unique_ID']
df = pd.read_csv(INPUT_FILE, usecols=cols_to_load)

# 2. Filter, Sort, and Pre-process
df = df[df['volume_ml'] >= 0].copy()
df = df.sort_values(['run_no', 'volume_ml']).reset_index(drop=True)
df['Fraction_unique_ID'] = df['Fraction_unique_ID'].replace('', np.nan)

# Helper: Moving Average Baseline Correction (Replaced Sparse ALS for speed)
def moving_average_baseline(y, window_fraction=0.05):
    window_size = max(11, int(len(y) * window_fraction))
    # Ensure window size is odd for uniform_filter1d
    if window_size % 2 == 0:
        window_size += 1
    return uniform_filter1d(y, size=window_size)

# Helper: Gaussian Fit
def gaussian(x, amplitude, mean, sigma):
    return amplitude * np.exp(-((x - mean) ** 2) / (2 * sigma ** 2))

# Helper: Calculate FWHM
def calculate_fwhm(sigma):
    return 2 * np.sqrt(2 * np.log(2)) * sigma

all_results = []
anomaly_count = 0
total_runs = 0
total_peaks = 0

for run_no, group in df.groupby('run_no'):
    total_runs += 1
    vol = group['volume_ml'].values
    signal_raw = group['UV_1_280_mAU'].values
    conc_b = group['Conc_B_%'].values
    frac_ids = group['Fraction_unique_ID'].values

    # Verify strictly increasing volume; aggregate duplicates if found
    if not np.all(np.diff(vol) > 0):
        temp_df = group[['volume_ml', 'UV_1_280_mAU', 'Conc_B_%',
                        'Fraction_unique_ID']].copy()
        agg_dict = {
            'UV_1_280_mAU': 'mean',
            'Conc_B_%': 'mean',
            'Fraction_unique_ID': lambda x: x.dropna().iloc[0] if len(x.dropna())
                > 0 else np.nan
        }
        temp_agg = temp_df.groupby('volume_ml', as_index=False).agg(agg_dict)
        vol = temp_agg['volume_ml'].values
        signal_raw = temp_agg['UV_1_280_mAU'].values
        conc_b = temp_agg['Conc_B_%'].values
        frac_ids = temp_agg['Fraction_unique_ID'].values

```

```

# 3. Baseline Correction (Moving Average - Faster)
baseline = moving_average_baseline(signal_raw)
signal_corrected = signal_raw - baseline

# Estimate Noise
window_size_noise = max(11, int(len(signal_corrected) * 0.01))
signal_smooth = uniform_filter1d(signal_corrected, size=window_size_noise)
high_freq_component = signal_corrected - signal_smooth
noise_std = np.std(high_freq_component)
if noise_std == 0:
    noise_std = 1e-6

# 4. Peak Detection: Direct derivative-based approach
deriv = np.gradient(signal_corrected, vol)
deriv_threshold = SNR_THRESHOLD * noise_std

# Apply find_peaks directly to the derivative
deriv_peaks, _ = find_peaks(deriv, height=deriv_threshold)

# Map resulting indices directly to the original signal's local maxima
signal_peaks, _ = find_peaks(signal_corrected, height=0.1 * noise_std)

matched_signal_peaks = []
if len(signal_peaks) > 0 and len(deriv_peaks) > 0:
    # Vectorized nearest-neighbor search using searchsorted
    # For each deriv_peak, find the closest signal_peak
    for dp in deriv_peaks:
        idx = np.searchsorted(signal_peaks, dp)
        candidates = []
        if idx < len(signal_peaks):
            candidates.append(signal_peaks[idx])
        if idx > 0:
            candidates.append(signal_peaks[idx-1])

        if candidates:
            closest = min(candidates, key=lambda x: abs(x - dp))
            matched_signal_peaks.append(closest)

matched_signal_peaks = sorted(list(set(matched_signal_peaks)))

peaks = np.array(matched_signal_peaks)

run_peaks = []

for i, peak_idx in enumerate(peaks):
    peak_height = signal_corrected[peak_idx]
    snr = peak_height / noise_std

    # 6. Filter SNR: Direct conditional check
    if snr <= SNR_THRESHOLD:
        anomaly_count += 1
        continue

# 5. Gaussian Fit
window_size = max(10, int(len(vol) * 0.05))
start_idx = max(0, peak_idx - window_size)
end_idx = min(len(vol), peak_idx + window_size)

```

```

x_fit = vol[start_idx:end_idx]
y_fit = signal_corrected[start_idx:end_idx]

try:
    p0 = [peak_height, vol[peak_idx], 1.0]
    popt, _ = curve_fit(gaussian, x_fit, y_fit, p0=p0, maxfev=1000)

    amplitude, mean_vol, sigma = popt
    fwhm = calculate_fwhm(sigma)
    auc = amplitude * sigma * np.sqrt(2 * np.pi)

    # 7. Handle Missing Fraction ID
    window_frac_ids = frac_ids[start_idx:end_idx]

    # Refactor empty window check
    if len(window_frac_ids) == 0:
        # Expand window indices
        expanded_start = max(0, peak_idx - window_size * 2)
        expanded_end = min(len(vol), peak_idx + window_size * 2)
        expanded_frac_ids = frac_ids[expanded_start:expanded_end]

        if len(expanded_frac_ids) == 0 or pd.isna(expanded_frac_ids).all():
            :
            has_fraction = False
        else:
            has_fraction = not pd.isna(expanded_frac_ids).all()
    else:
        has_fraction = not (pd.isna(window_frac_ids).all())

    fraction_status = 'Fractionated' if has_fraction else 'Non-
        Fractionated'

    # Count anomaly if missing ID
    if not has_fraction:
        anomaly_count += 1

    # Vectorized search for closest_idx
    idx = np.searchsorted(vol, mean_vol)
    if idx == 0:
        closest_idx = 0
    elif idx == len(vol):
        closest_idx = len(vol) - 1
    else:
        left_val = vol[idx-1]
        right_val = vol[idx]
        closest_idx = idx-1 if np.abs(left_val - mean_vol) < np.abs(
            right_val - mean_vol) else idx

    conc_b_val = conc_b[closest_idx]

    run_peaks.append({
        'run_no': run_no,
        'peak_id': i + 1,
        'retention_volume_ml': mean_vol,
        'peak_height_mAU': peak_height,
        'peak_area_mAU_ml': auc,
        'fwhm_ml': fwhm,
        'snr': snr,
        'conc_b_percent': conc_b_val,

```

```

        'fraction_status': fraction_status
    })

except RuntimeError:
    # Check for missing IDs before incrementing anomaly count
    # Re-evaluate fraction status for this peak if fit failed
    window_size = max(10, int(len(vol) * 0.05))
    start_idx = max(0, peak_idx - window_size)
    end_idx = min(len(vol), peak_idx + window_size)
    window_frac_ids = frac_ids[start_idx:end_idx]

    if len(window_frac_ids) == 0:
        expanded_start = max(0, peak_idx - window_size * 2)
        expanded_end = min(len(vol), peak_idx + window_size * 2)
        expanded_frac_ids = frac_ids[expanded_start:expanded_end]
        has_fraction = not (len(expanded_frac_ids) == 0 or pd.isna(
            expanded_frac_ids).all())
    else:
        has_fraction = not (pd.isna(window_frac_ids).all())

    if not has_fraction:
        anomaly_count += 1
    # If fit failed but ID exists, it's still an anomaly (fit failure)
    anomaly_count += 1
    continue

# Check if Fraction_unique_ID is missing for the entire run
if pd.isna(frac_ids).all():
    for peak in run_peaks:
        peak['fraction_status'] = 'Non-Fractionated'

total_peaks += len(run_peaks)
all_results.extend(run_peaks)

# 8. Generate Summary and Output
avg_peak_count = total_peaks / total_runs if total_runs > 0 else 0

output_lines = []
output_lines.append("#_Peak_Catalog_Summary\n")
output_lines.append("|_Metric_|_Value_|")
output_lines.append("|---|---|")
output_lines.append(f"|_Total_Runs_|_{total_runs}_|")
output_lines.append(f"|_Avg_Peak_Count_|_{avg_peak_count:.2f}_|")
output_lines.append(f"|_Anomaly_Count_|_{anomaly_count}_|")
output_lines.append("\n")

results_df = pd.DataFrame(all_results)
if not results_df.empty:
    # Strictly use groupby
    for run_no, run_data in results_df.groupby('run_no'):
        output_lines.append(f"##_Run_{run_no}\n")
        output_lines.append(f"|_run_no_|_peak_id_|_retention_volume_ml_|_
            peak_height_mAU_|_peak_area_mAU_ml_|_fwhm_ml_|_snr_|_conc_b_percent_|_
            fraction_status_|")
        output_lines.append("|---|---|---|---|---|---|---|---|---|")

        limited_data = run_data.head(MAX_OUTPUT_LINES_PER_RUN)

    # Use apply for row formatting

```



```

if missing_cols:
    logger.error(f"Missing required columns in input file: {missing_cols}")
    with open(OUTPUT_FILE, 'a') as f:
        f.write("## Error\n")
        f.write(f"Missing required columns in input file: {missing_cols}\n")
    sys.exit(1)

# Load only the required columns
df = pd.read_csv(INPUT_FILE, usecols=required_cols)

# Filter Invalid Flow Rates: Remove rows where System_flow_ml_min <= 0 or is
# NaN
initial_rows = len(df)
df = df[(df['System_flow_ml_min'] > 0) & (df['System_flow_ml_min'].notna())]
dropped_rows = initial_rows - len(df)
if dropped_rows > 0:
    logger.info(f"Dropped {dropped_rows} rows due to invalid or missing
                System_flow_ml_min values.")

except FileNotFoundError:
    logger.error(f"Error: {INPUT_FILE} not found.")
    with open(OUTPUT_FILE, 'a') as f:
        f.write("## Error\n")
        f.write(f"Input file {INPUT_FILE} not found.\n")
    sys.exit(1)

# 2. Derive peak_id and peak_area from raw signal
logger.info("Deriving peak_id and peak_area from UV_1_280_mAU signal...")

def detect_peaks_and_areas(group):
    """
    Detects peaks in the UV signal for a specific run and calculates their area.
    Returns a DataFrame with peak_id and peak_area for each detected peak.
    """
    if group.empty:
        return pd.DataFrame(columns=['peak_id', 'peak_area', 'norm_volume_ml', '
            Conc_B_%',
                                     'fraction_volume_ml_min_precise', '
                                     fraction_volume_ml_max_precise'])

    # Sort by volume to ensure correct order
    group = group.sort_values('volume_ml').reset_index(drop=True)

    signal = group['UV_1_280_mAU'].values
    volumes = group['volume_ml'].values
    conc_b = group['Conc_B_%'].values
    flow_rate = group['System_flow_ml_min'].iloc[0]

    # Safe Division in Peak Detection: Check if flow_rate is valid
    if not np.isfinite(flow_rate) or flow_rate == 0:
        logger.warning(f"Invalid flow rate {flow_rate} for run {group['run_no']}.
            iloc[0]}. Skipping peak normalization.")
        # Return empty or handle gracefully; here we return empty to avoid
        # propagation
        return pd.DataFrame(columns=['peak_id', 'peak_area', 'norm_volume_ml', '
            Conc_B_%',
                                     'fraction_volume_ml_min_precise', '
                                     fraction_volume_ml_max_precise'])

```

```

# Find peaks using scipy
# height threshold can be adjusted; here we use a small fraction of max to
# avoid noise
height_threshold = 0.05 * np.max(signal)
peaks, properties = find_peaks(signal, height=height_threshold, distance=5)

if len(peaks) == 0:
    return pd.DataFrame(columns=['peak_id', 'peak_area', 'norm_volume_ml', '
        Conc_B_%',
                                'fraction_volume_ml_min_precise', '
                                fraction_volume_ml_max_precise'])

peak_data = []
for i, peak_idx in enumerate(peaks):
    peak_id = i + 1
    peak_vol = volumes[peak_idx]
    # Safe division ensured by check above
    peak_norm_vol = peak_vol / flow_rate
    peak_conc_b = conc_b[peak_idx]

    # Calculate area: simple trapezoidal integration around the peak
    start_idx = max(0, peak_idx - 20)
    end_idx = min(len(signal), peak_idx + 20)

    peak_height = signal[peak_idx]
    area = np.trapz(signal[start_idx:end_idx], volumes[start_idx:end_idx])

    # Get fraction boundaries for this peak (approximate using the row at
    # peak_idx)
    frac_min = group['fraction_volume_ml_min_precise'].iloc[peak_idx]
    frac_max = group['fraction_volume_ml_max_precise'].iloc[peak_idx]

    peak_data.append({
        'peak_id': peak_id,
        'peak_area': area,
        'norm_volume_ml': peak_norm_vol,
        'Conc_B_%': peak_conc_b,
        'fraction_volume_ml_min_precise': frac_min,
        'fraction_volume_ml_max_precise': frac_max,
        'run_no': group['run_no'].iloc[0]
    })

return pd.DataFrame(peak_data)

# Apply peak detection per run
peak_dfs = []
for run_id, group in df.groupby('run_no'):
    peak_df = detect_peaks_and_areas(group)
    if not peak_df.empty:
        peak_dfs.append(peak_df)

if not peak_dfs:
    logger.error("No peaks detected in any run.")
    with open(OUTPUT_FILE, 'a') as f:
        f.write("##Error\n")
        f.write("No peaks detected in any run.\n")
    sys.exit(1)

df_peaks = pd.concat(peak_dfs, ignore_index=True)

```

```

logger.info(f"Detected_{len(df_peaks)}_peaks_across_{df_peaks['run_no'].nunique()}_runs.")

# 3. Pre-process missing values (if any in derived data)
for col in ['fraction_volume_ml_min_precise', 'fraction_volume_ml_max_precise']:
    df_peaks[col] = df_peaks.groupby('run_no')[col].transform(lambda x: x.fillna(x.median()))

df_peaks = df_peaks.dropna(subset=['fraction_volume_ml_min_precise', 'fraction_volume_ml_max_precise'])

# 4. Match peaks across runs
# Recalculate fraction width and normalized width based on derived data
df_peaks['fraction_width_ml'] = df_peaks['fraction_volume_ml_max_precise'] - df_peaks['fraction_volume_ml_min_precise']
# We need flow rate for normalization. It's not in df_peaks directly, so we merge back or assume constant per run.
# Let's merge flow rate from original df
flow_rates = df.groupby('run_no')['System_flow_ml_min'].first().reset_index()
flow_rates.columns = ['run_no', 'System_flow_ml_min']
df_peaks = df_peaks.merge(flow_rates, on='run_no', how='left')

# Safe Division in Normalization: Replace any remaining zero/NaN flow rates with the median flow rate of that specific run
# Although df was filtered, merge might introduce NaNs if run_no mismatched (unlikely here) or if logic changed.
# We ensure safety by filling NaNs with the median of the run's flow rate.
median_flow_per_run = df.groupby('run_no')['System_flow_ml_min'].median().reset_index()
median_flow_per_run.columns = ['run_no', 'median_flow_ml_min']
df_peaks = df_peaks.merge(median_flow_per_run, on='run_no', how='left')

# Fill invalid flow rates with the run's median
invalid_flow_mask = df_peaks['System_flow_ml_min'].isna() | (df_peaks['System_flow_ml_min'] <= 0)
if invalid_flow_mask.any():
    logger.info(f"Replacing_{invalid_flow_mask.sum()}_invalid_flow_rates_with_run_median.")
    df_peaks.loc[invalid_flow_mask, 'System_flow_ml_min'] = df_peaks.loc[invalid_flow_mask, 'median_flow_ml_min']

df_peaks['norm_fraction_width'] = df_peaks['fraction_width_ml'] / df_peaks['System_flow_ml_min']
global_mean_fwhm = df_peaks['norm_fraction_width'].mean()
tolerance = TOLERANCE_FACTOR * global_mean_fwhm

# Verify Tolerance: Log values to confirm they are finite
if not np.isfinite(global_mean_fwhm) or not np.isfinite(tolerance):
    logger.error(f"Calculated_global_mean_fwhm_{global_mean_fwhm}_or_tolerance_{tolerance}_is_not_finite.")
    with open(OUTPUT_FILE, 'a') as f:
        f.write("##_Error\n")
        f.write(f"Calculated_global_mean_fwhm_{global_mean_fwhm}_or_tolerance_{tolerance}_is_not_finite.\n")
    sys.exit(1)

logger.info(f"Global_Mean_FWHM:{global_mean_fwhm},_Tolerance:{tolerance}")

# 5. Filter for high intensity and consistent presence

```

```

intensity_threshold = df_peaks['peak_area'].quantile(INTENSITY_PERCENTILE / 100.0)
high_intensity_df = df_peaks[df_peaks['peak_area'] >= intensity_threshold].copy()

logger.info(f"Number of peaks remaining after intensity filter (>{
    INTENSITY_PERCENTILE}th percentile): {len(high_intensity_df)}")

if len(high_intensity_df) == 0:
    logger.warning("No high intensity peaks found.")
    with open(OUTPUT_FILE, 'a') as f:
        f.write("## Step 2: Target Identification and Statistical Elution Window\n")
        f.write("No high intensity peaks found.\n")
    sys.exit(1)

# Use DBSCAN to group peaks
X = high_intensity_df[['norm_volume_ml']].values
dbscan = DBSCAN(eps=tolerance, min_samples=2)
dbscan.fit(X)
high_intensity_df['peak_group'] = dbscan.labels_

# Debug Logging for DBSCAN groups
group_run_counts = high_intensity_df.groupby('peak_group')['run_no'].nunique()
total_runs = df_peaks['run_no'].nunique()
logger.info(f"Number of unique peak groups found by DBSCAN: {len(group_run_counts)}")
logger.info(f"Run counts per group: {group_run_counts.to_dict()}")

valid_groups = group_run_counts[group_run_counts >= (total_runs *
    PRESENCE_THRESHOLD)].index
high_intensity_consistent_df = high_intensity_df[high_intensity_df['peak_group'].
    isin(valid_groups)].copy()

# 6. Define clustering feature space and filter outliers using Conc_B_%
if high_intensity_consistent_df.empty:
    logger.warning("No consistent peaks found across runs.")
    with open(OUTPUT_FILE, 'a') as f:
        f.write("## Step 2: Target Identification and Statistical Elution Window\n")
        f.write("No consistent peaks found across runs.\n")
    sys.exit(1)

conc_b_mean = high_intensity_consistent_df['Conc_B_%'].mean()
conc_b_std = high_intensity_consistent_df['Conc_B_%'].std()
conc_b_lower = conc_b_mean - 3 * conc_b_std
conc_b_upper = conc_b_mean + 3 * conc_b_std

filtered_df = high_intensity_consistent_df[
    (high_intensity_consistent_df['Conc_B_%'] >= conc_b_lower) &
    (high_intensity_consistent_df['Conc_B_%'] <= conc_b_upper)
].copy()

# 7. Apply DBSCAN to identify dominant target cluster
if len(filtered_df) == 0:
    logger.warning("No peaks remaining after filtering.")
    with open(OUTPUT_FILE, 'a') as f:
        f.write("## Step 2: Target Identification and Statistical Elution Window\n")
        f.write("No peaks remaining after filtering.\n")
    sys.exit(1)

```

```

X_filtered = filtered_df[['norm_volume_ml']].values
dbscan_final = DBSCAN(eps=tolerance, min_samples=2)
dbscan_final.fit(X_filtered)
filtered_df['cluster_id'] = dbscan_final.labels_

if -1 in filtered_df['cluster_id'].values:
    cluster_counts = filtered_df[filtered_df['cluster_id'] != -1]['cluster_id'].
        value_counts()
    if len(cluster_counts) == 0:
        logger.warning("No valid clusters found.")
        with open(OUTPUT_FILE, 'a') as f:
            f.write("## Step 2: Target Identification and Statistical Elution Window\n")
            f.write("No valid clusters found.\n")
        sys.exit(1)
    target_cluster_id = cluster_counts.idxmax()
else:
    cluster_counts = filtered_df['cluster_id'].value_counts()
    target_cluster_id = cluster_counts.idxmax()

target_df = filtered_df[filtered_df['cluster_id'] == target_cluster_id].copy()

if target_df.empty:
    logger.error("Target cluster is empty.")
    with open(OUTPUT_FILE, 'a') as f:
        f.write("## Step 2: Target Identification and Statistical Elution Window\n")
        f.write("Target cluster is empty.\n")
    sys.exit(1)

target_norm_volume_mean = target_df['norm_volume_ml'].mean()
target_norm_volume_std = target_df['norm_volume_ml'].std()

# 8. Algorithmically derive fraction boundaries (Vectorized)
def calculate_boundaries_vectorized(run_data):
    run_data = run_data.sort_values('fraction_volume_ml_min_precise').reset_index(
        drop=True)
    if run_data.empty:
        return [], []

    starts = run_data['fraction_volume_ml_min_precise'].values
    ends = run_data['fraction_volume_ml_max_precise'].values

    current_end = ends[0]
    merged_starts = [starts[0]]
    merged_ends = [current_end]

    for i in range(1, len(starts)):
        if starts[i] <= current_end:
            current_end = max(current_end, ends[i])
            merged_ends[-1] = current_end
        else:
            merged_starts.append(starts[i])
            merged_ends.append(ends[i])
            current_end = ends[i]

    merged_ranges = list(zip(merged_starts, merged_ends))

```

```

dead_time_ranges = []
inter_fraction_ranges = []

if merged_ranges:
    if merged_ranges[0][0] > 0:
        dead_time_ranges.append((0, merged_ranges[0][0]))

    for i in range(len(merged_ranges) - 1):
        gap_start = merged_ranges[i][1]
        gap_end = merged_ranges[i+1][0]
        if gap_end > gap_start:
            inter_fraction_ranges.append((gap_start, gap_end))

    return dead_time_ranges, inter_fraction_ranges

# 9. Categorize peaks
if len(filtered_df) == 0:
    logger.error("Cannot calculate gradient range: filtered_df is empty.")
    with open(OUTPUT_FILE, 'a') as f:
        f.write("## Step 2: Target Identification and Statistical Elution Window\n")
        f.write("Error: Cannot calculate gradient range.\n")
    sys.exit(1)

gradient_conc_min = filtered_df['Conc_B_%'].min()
gradient_conc_max = filtered_df['Conc_B_%'].max()

target_window_lower = target_norm_volume_mean - 2 * target_norm_volume_std
target_window_upper = target_norm_volume_mean + 2 * target_norm_volume_std

categorized_df = high_intensity_consistent_df.copy()
categorized_df['category'] = 'Potential Impurity'

mask_target = (categorized_df['norm_volume_ml'] >= target_window_lower) & \
    (categorized_df['norm_volume_ml'] <= target_window_upper)
categorized_df.loc[mask_target, 'category'] = 'Target'

# Calculate boundaries per run
run_boundaries = {}
for run_id in df['run_no'].unique():
    run_data = df[df['run_no'] == run_id]
    if run_data.empty:
        continue
    flow_rate = run_data['System_flow_ml_min'].iloc[0]
    if not np.isfinite(flow_rate) or flow_rate == 0:
        continue
    dead_ml, inter_ml = calculate_boundaries_vectorized(run_data)
    dead_norm = [(s/flow_rate, e/flow_rate) for s, e in dead_ml]
    inter_norm = [(s/flow_rate, e/flow_rate) for s, e in inter_ml]
    run_boundaries[run_id] = (dead_norm, inter_norm)

# Vectorized anomaly detection
mask_outside_gradient = (categorized_df['Conc_B_%'] < gradient_conc_min) | \
    (categorized_df['Conc_B_%'] > gradient_conc_max)

mask_in_dead_inter = pd.Series(False, index=categorized_df.index)

if not run_boundaries:
    logger.warning("No run boundaries calculated. Skipping boundary check.")

```

```

else:
    for run_id, (dead_norm, inter_norm) in run_boundaries.items():
        run_mask = categorized_df['run_no'] == run_id
        if not run_mask.any():
            continue

        run_subset = categorized_df.loc[run_mask]
        run_vols = run_subset['norm_volume_ml'].values

        in_dead = np.zeros(len(run_vols), dtype=bool)
        for start, end in dead_norm:
            in_dead |= (run_vols >= start) & (run_vols <= end)

        in_inter = np.zeros(len(run_vols), dtype=bool)
        for start, end in inter_norm:
            in_inter |= (run_vols >= start) & (run_vols <= end)

        mask_in_dead_inter.loc[run_mask] = in_dead | in_inter

mask_anomaly = mask_outside_gradient | mask_in_dead_inter
categorized_df.loc[mask_anomaly & (~mask_target), 'category'] = 'Critical_Anomaly'

# Prepare Output
output_lines = []
output_lines.append("##_Step_2:_Target_Identification_and_Statistical_Elution_
    Window\n")
output_lines.append("**Global_Target_Statistics**:\n")
output_lines.append(f"-_Target_Norm_Volume_Mean:_{{target_norm_volume_mean:.4f}}\n")
output_lines.append(f"-_Target_Norm_Volume_Std:_{{target_norm_volume_std:.4f}}\n")
output_lines.append(f"-_Target_Window_(Mean_ _2_ ):_{{target_window_lower:.4f}},_{{
    target_window_upper:.4f}}\n")
output_lines.append(f"-_Gradient_Range_(Conc_B_%):_{{gradient_conc_min:.2f}},_{{
    gradient_conc_max:.2f}}\n")
output_lines.append("\n**Per-Run_Summary**:\n")

# Group by run_no for the summary
for run_id, run_data in categorized_df.groupby('run_no'):
    run_summary = []
    run_summary.append(f"###_Run_{run_id}")
    run_summary.append(f"-_target_norm_volume_mean:_{{target_norm_volume_mean:.4f}}")
    run_summary.append(f"-_target_norm_volume_std:_{{target_norm_volume_std:.4f}}")
    run_summary.append(f"-_anomaly_peaks:")

    all_issues = run_data[run_data['category'].isin(['Critical_Anomaly', '
        Potential_Impurity'])]

    if len(all_issues) > 0:
        all_issues = all_issues.sort_values('norm_volume_ml')
        count = 0
        for _, row in all_issues.iterrows():
            if count >= 20:
                run_summary.append(f"_..._({len(all_issues)}-_20_+more)")
                break
            run_summary.append(f"_Category:_{{row['category']}},_Retention_Volume
                :_{{row['norm_volume_ml']:.4f}}")
            count += 1
    else:
        run_summary.append("_None")

```

```

        output_lines.extend(run_summary)
        output_lines.append("")

# Write to file (Append mode)
with open(OUTPUT_FILE, 'a') as f:
    f.write('\n'.join(output_lines))

logger.info(f"Analysis complete. Results appended to {OUTPUT_FILE}")

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import correlate
from scipy.interpolate import interp1d
import warnings

# Define paths
input_file = '/inputs/chromatography_combined.csv'
output_file = '/workspace/purity_correlation.png'

# Load data
df = pd.read_csv(input_file, on_bad_lines='skip')

# 1. Inspect data immediately after loading
print(f"Dataframe shape: {df.shape}")
print(f"First 5 rows of critical columns:")
cols_to_inspect = ['volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU']
for col in cols_to_inspect:
    if col in df.columns:
        print(f"Column '{col}':\n{df[col].head()}")

# Inspect raw columns 30 and 31 by index if they exist
if len(df.columns) > 30:
    print(f"Raw column 30 (index 30):\n{df.iloc[:, 30].head()}")
if len(df.columns) > 31:
    print(f"Raw column 31 (index 31):\n{df.iloc[:, 31].head()}")

# 2. Clean data strategy: Coerce then Fill (instead of Drop)
numeric_cols_to_clean = ['volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU']

for col in numeric_cols_to_clean:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors='coerce')
        df[col] = df[col].fillna(0)

# 3. Check if dataframe is empty after cleaning
if len(df) == 0:
    raise ValueError("Dataframe is empty after cleaning. Check raw data inspection output.")

# Verify dtypes after cleaning
print(f"Data types after cleaning: {df[['volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU']].dtypes}")
print(f"Unique values in volume_ml (sample): {df['volume_ml'].unique()[:10]}")

# Ensure required columns exist

```

```

required_cols = ['run_no', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU']
if not all(col in df.columns for col in required_cols):
    raise ValueError(f"Missing required columns. Available: {df.columns.tolist()}")
)

# Step 0: Implement Peak Detection to replace synthetic peak_id
# Calculate baseline (median) and threshold
uv_280_signal = df['UV_1_280_mAU'].values
baseline = np.median(uv_280_signal)
# Threshold: baseline + small offset (e.g., 5% of max signal or fixed small value)
max_signal = np.max(uv_280_signal)
threshold = baseline + max(0.05 * max_signal, 0.1)

# Identify indices where signal is above threshold
above_threshold = uv_280_signal > threshold

# Group consecutive indices into peak_id clusters
peak_id = np.zeros(len(df), dtype=int)
current_peak = -1
in_peak = False

for i, is_above in enumerate(above_threshold):
    if is_above:
        if not in_peak:
            current_peak += 1
            in_peak = True
            peak_id[i] = current_peak
        else:
            in_peak = False

# Assign new peak_id to dataframe
df['peak_id'] = peak_id

# Step 1: Align signals using cross-correlation
uv_280 = df['UV_1_280_mAU'].values
uv_260 = df['UV_2_260_mAU'].values

# Normalize for cross-correlation
uv_280_norm = (uv_280 - np.mean(uv_280)) / np.std(uv_280)
uv_260_norm = (uv_260 - np.mean(uv_260)) / np.std(uv_260)

# Compute cross-correlation
corr = correlate(uv_260_norm, uv_280_norm, mode='full')
lag = np.argmax(corr) - (len(uv_280) - 1)

# Apply fixed offset if inconclusive
if abs(lag) > len(uv_280) * 0.05:
    lag = 0

# Shift UV_260 to align with UV_280 using direct array slicing
if lag != 0:
    if lag > 0:
        uv_260_shifted = np.concatenate((np.full(lag, np.nan), uv_260[:-lag]))
    else:
        uv_260_shifted = np.concatenate((uv_260[:lag], np.full(-lag, np.nan)))
else:
    uv_260_shifted = uv_260.copy()

df['UV_2_260_mAU_aligned'] = uv_260_shifted

```

```

# Step 2: Prepare peak data for vectorized AUC calculation
# Sort by peak_id to facilitate splitting
df_sorted = df.sort_values('peak_id').reset_index(drop=True)

# Identify group boundaries
peak_ids = df_sorted['peak_id'].values
# Find indices where peak_id changes
change_indices = np.where(np.diff(peak_ids) != 0)[0] + 1
# Include the start and end
split_indices = np.concatenate(([0], change_indices, [len(df_sorted)]))

# List to store results
peak_results = []

# Iterate through groups defined by split_indices
for start, end in zip(split_indices[:-1], split_indices[1:]):
    group = df_sorted.iloc[start:end]

    # Extract metadata
    peak_id_val = group['peak_id'].iloc[0]
    run_no = group['run_no'].iloc[0]
    volumes = group['volume_ml'].values
    uv_280_vals = group['UV_1_280_mAU'].values
    uv_260_vals = group['UV_2_260_mAU_aligned'].values

    # Check for sufficient valid points
    mask_280 = ~np.isnan(uv_280_vals)
    mask_260 = ~np.isnan(uv_260_vals)

    if np.sum(mask_280) < 2 or np.sum(mask_260) < 2:
        continue

    # Interpolate missing values
    if not np.all(mask_280):
        interp_func_280 = interp1d(volumes[mask_280], uv_280_vals[mask_280], kind=
            'linear', fill_value='extrapolate')
        uv_280_interp = interp_func_280(volumes)
    else:
        uv_280_interp = uv_280_vals

    if not np.all(mask_260):
        interp_func_260 = interp1d(volumes[mask_260], uv_260_vals[mask_260], kind=
            'linear', fill_value='extrapolate')
        uv_260_interp = interp_func_260(volumes)
    else:
        uv_260_interp = uv_260_vals

    # Calculate AUC
    auc_280 = np.trapz(uv_280_interp, volumes)
    auc_260 = np.trapz(uv_260_interp, volumes)
    ratio = auc_260 / (auc_280 + 1e-9)

    peak_results.append({
        'peak_id': peak_id_val,
        'run_no': run_no,
        'AUC_280': auc_280,
        'AUC_260': auc_260,
        'Ratio_260_280': ratio
    })

```

```

    })

peak_data = pd.DataFrame(peak_results)

if peak_data.empty:
    raise ValueError("No valid peaks found after processing.")

# Step 3: Flag potential nucleic acid contaminants
threshold = peak_data['Ratio_260_280'].quantile(0.9)
peak_data['is_contaminant'] = peak_data['Ratio_260_280'] > threshold

# Step 4: Visualize correlation (Plotting aggregated peak metrics as per feedback
R03)
# Prepare scatter plot data
scatter_df = peak_data[peak_data['AUC_280'] > 0].copy()

# Determine if density overlay is needed
if len(scatter_df) > 500:
    # Use hexbin or scatter with alpha/s scaling
    plt.figure(figsize=(10, 8))
    unique_runs = scatter_df['run_no'].unique()
    colors = plt.cm.tab10(np.linspace(0, 1, len(unique_runs)))

    for i, run in enumerate(unique_runs):
        run_data = scatter_df[scatter_df['run_no'] == run]
        if len(run_data) > 0:
            # Use scatter with alpha and small size for density effect
            plt.scatter(
                run_data['AUC_280'],
                run_data['AUC_260'],
                c=[colors[i % len(colors)]],
                marker='o',
                label=f'Run_{run}',
                alpha=0.1,
                s=1,
                edgecolors='none'
            )
else:
    # If <= 500 points, plot normally
    plt.figure(figsize=(10, 8))
    unique_runs = scatter_df['run_no'].unique()
    colors = plt.cm.tab10(np.linspace(0, 1, len(unique_runs)))
    markers = ['o', 's', '^', 'D', 'v', 'p', '*', 'h', 'H', '+']

    for i, run in enumerate(unique_runs):
        run_data = scatter_df[scatter_df['run_no'] == run]
        plt.scatter(
            run_data['AUC_280'],
            run_data['AUC_260'],
            c=[colors[i % len(colors)]],
            marker=markers[i % len(markers)],
            label=f'Run_{run}',
            alpha=0.7,
            edgecolors='k',
            linewidth=0.5
        )

plt.xlabel('AUC_280 (Protein Area)')
plt.ylabel('AUC_260 (Nucleic Acid Area)')

```

```
plt.title('Protein_vs_Nucleic_Acid_Signal_Correlation_by_Run')

# Legend
if len(unique_runs) > 10:
    plt.legend(bbox_to_anchor=(1.05, 1), loc='upper_left')
else:
    plt.legend()

plt.grid(True, linestyle='--', alpha=0.3)
plt.tight_layout()
plt.savefig(output_file, dpi=300, bbox_inches='tight')
plt.close()

print(f"Plot_saved_to_{output_file}")
```

Listing 3: Code_3